ASSIGNMENT-16.4

2303A51304

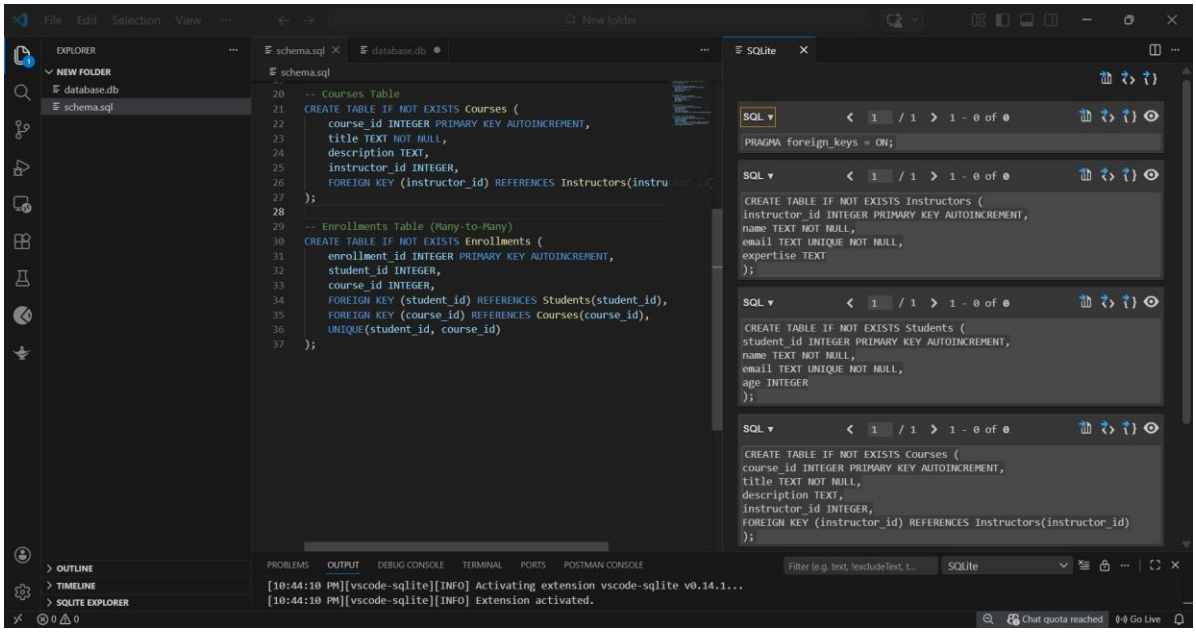
Batch-05

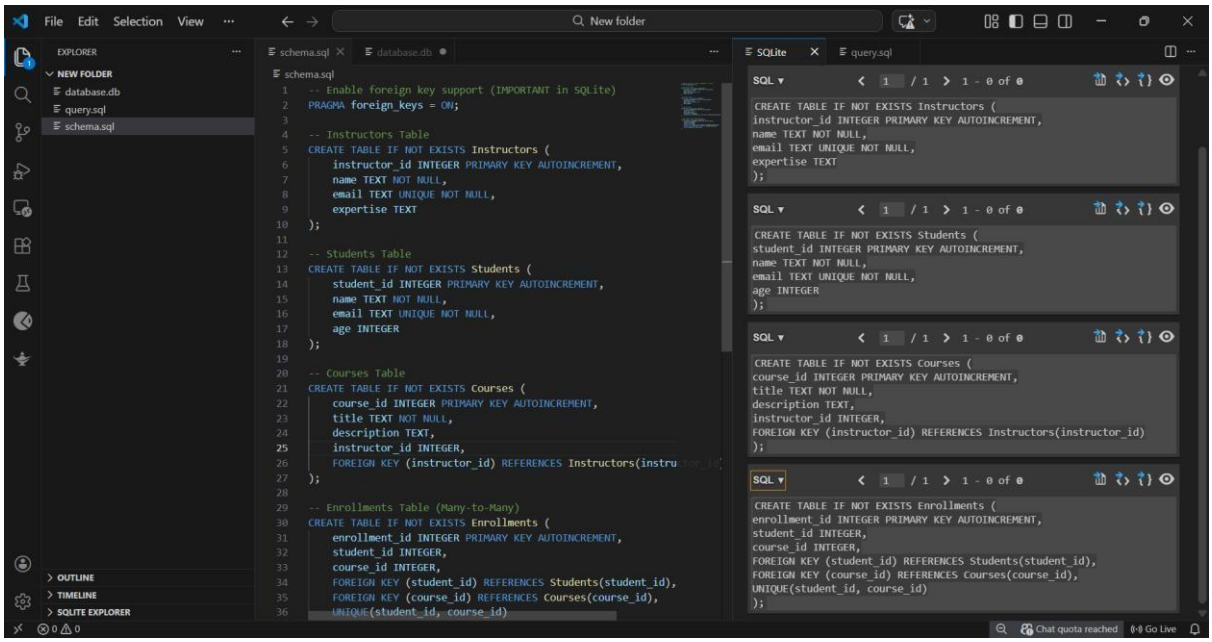
Task 1:

- **Prompt:**

```
Generate SQL CREATE TABLE statements for an online course platform.
Tables: Instructors, Courses, Students.
Include primary keys, foreign keys, and proper data types.
Ensure normalization.
```

Code:





Explanation:

- The schema uses separate tables (Instructors, Students, Courses) to avoid data redundancy.
- A many-to-many relationship between students and courses is managed using the Enrollments table.
- Primary keys uniquely identify each record, and foreign keys maintain relationships between tables.

- The design follows normalization principles, ensuring data consistency and integrity.

Task 2: Populate the Database with Sample Records
Scenario

To test the system, the platform needs initial sample data.

Task Description

- Use AI assistance to generate INSERT statements that add:

o At least 3 instructors

o At least 4 courses o

At least 3 students

- Verify correctness using SELECT queries.

Prompt:

```
Generate SQL INSERT statements for:
3 instructors, 4 courses, 3 students.
```

Code:

File Edit Selection View ··· New folder

EXPLORER
NEW FOLDER
database.db
insert.sql
query.sql
schema.sql
verify.sql

schema.sql

```sql
-- Enable foreign key support (IMPORTANT in SQLite)
PRAGMA foreign_keys = ON;

-- Instructors Table
CREATE TABLE IF NOT EXISTS Instructors (
    instructor_id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    email TEXT UNIQUE NOT NULL,
    expertise TEXT
);

-- Students Table
CREATE TABLE IF NOT EXISTS Students (
    student_id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    email TEXT UNIQUE NOT NULL,
    age INTEGER
);

-- Courses Table
CREATE TABLE IF NOT EXISTS Courses (
    course_id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT NOT NULL,
    description TEXT,
    instructor_id INTEGER,
    FOREIGN KEY (instructor_id) REFERENCES Instructors(instru
);

-- Enrollments Table (Many-to-Many)
CREATE TABLE IF NOT EXISTS Enrollments (
    enrollment_id INTEGER PRIMARY KEY AUTOINCREMENT,
    student_id INTEGER,
    course_id INTEGER,
    FOREIGN KEY (student_id) REFERENCES Students(student_id),
    FOREIGN KEY (course_id) REFERENCES Courses(course_id),
    UNIQUE(student_id, course_id)
```

query.sql

```sql
-- Instructors
INSERT INTO Instructors (name, email, expertise) VALUES
('Amina Patel', 'amina.patel@example.com', 'Data Science'),
('Diego Martinez', 'diego.martinez@example.com', 'Web Develop
('Lina Chen', 'lina.chen@example.com', 'UI/UX Design');

-- Students
INSERT INTO Students (name, email, age) VALUES
('Noah Kim', 'noah.kim@example.com', 20),
('Priya Rao', 'priya.rao@example.com', 21),
('Marcus Taylor', 'marcus.taylor@example.com', 22);

-- Courses
INSERT INTO Courses (title, description, instructor_id) VALUE
('Python Basics', 'Intro to Python', 1),
('Machine Learning', 'ML concepts', 1),
('React Development', 'Frontend with React', 2),
('UI/UX Design', 'Design principles', 3);
```

OUTLINE
TIMELINE
SQLITE EXPLORER

Ln 10, Col 25 Spaces: 4 UTF-8 CRLF {} SQLite Chat quota reached SQLite: database.db Go Live

File Edit Selection View ··· New folder

EXPLORER
NEW FOLDER
database.db
insert.sql
query.sql
schema.sql
verify.sql

schema.sql

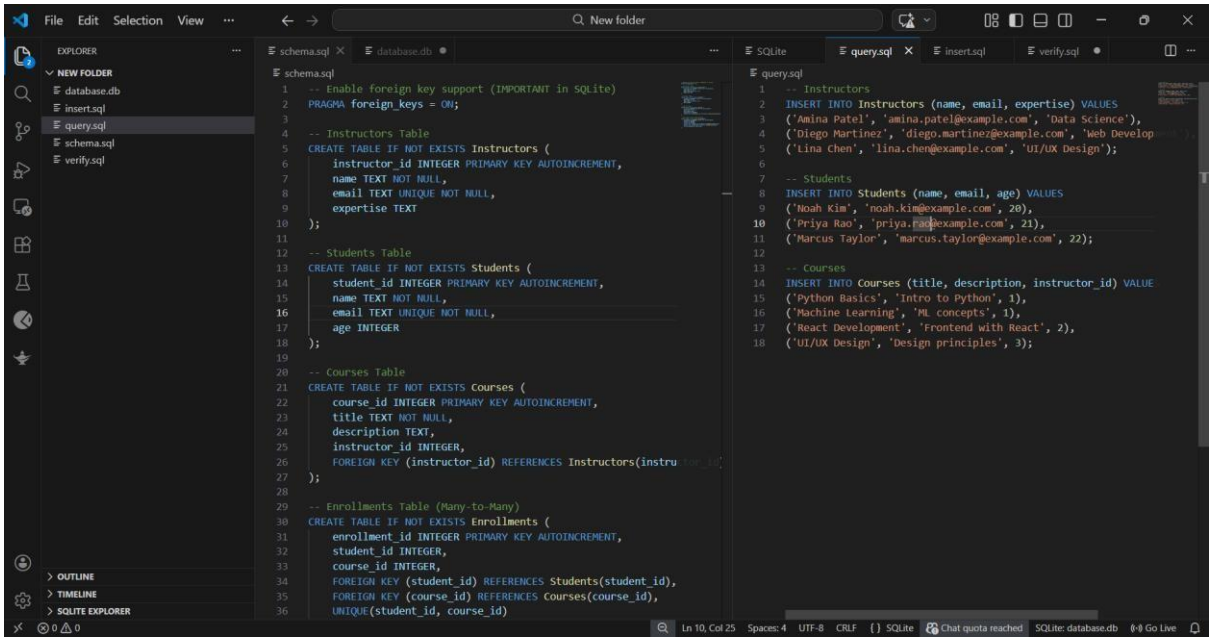
```sql
-- Enable foreign key support (IMPORTANT in SQLite)
PRAGMA foreign_keys = ON;

-- Instructors Table
CREATE TABLE IF NOT EXISTS Instructors (
    instructor_id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    email TEXT UNIQUE NOT NULL,
    expertise TEXT
);

-- Students Table
CREATE TABLE IF NOT EXISTS Students (
    student_id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    email TEXT UNIQUE NOT NULL,
    age INTEGER
);

-- Courses Table
CREATE TABLE IF NOT EXISTS Courses (
    course_id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT NOT NULL,
    description TEXT,
    instructor_id INTEGER,
    FOREIGN KEY (instructor_id) REFERENCES Instructors(instru
);

-- Enrollments Table (Many-to-Many)
CREATE TABLE IF NOT EXISTS Enrollments (
    enrollment_id INTEGER PRIMARY KEY AUTOINCREMENT,
    student_id INTEGER,
    course_id INTEGER,
    FOREIGN KEY (student_id) REFERENCES Students(student_id),
    FOREIGN KEY (course_id) REFERENCES Courses(course_id),
    UNIQUE(student_id, course_id)
```

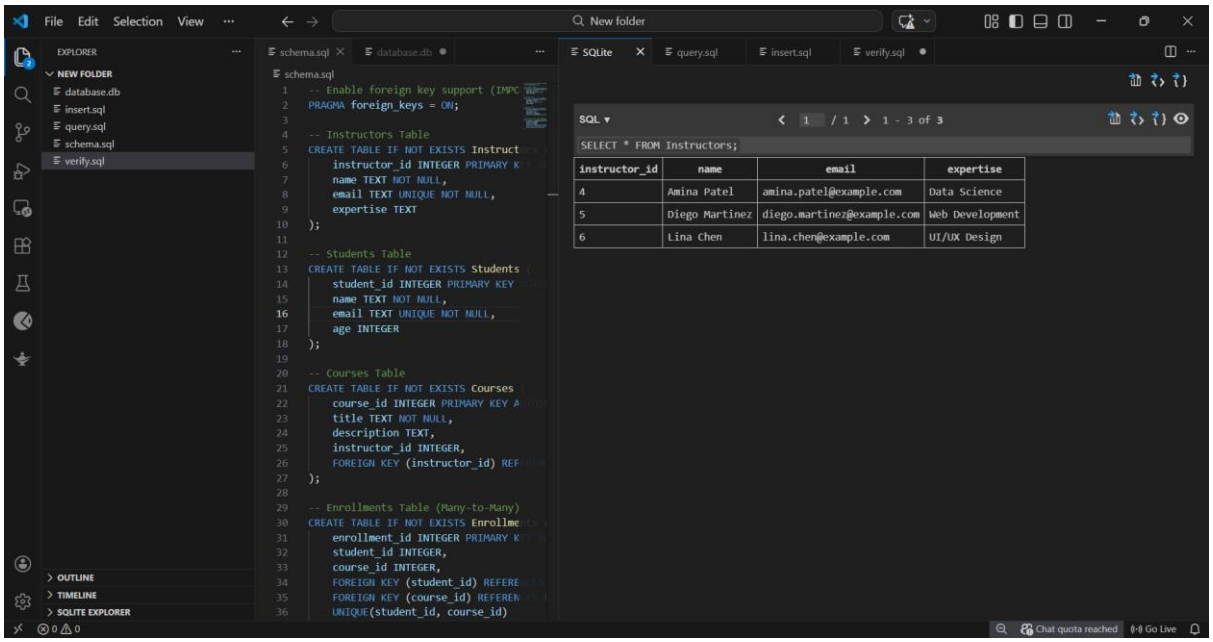
insert.sql

```sql
-- Instructors (avoid duplicate email error)
INSERT OR IGNORE INTO Instructors (name, email, expertise) VA
('Amina Patel', 'amina.patel@example.com', 'Data Science'),
('Diego Martinez', 'diego.martinez@example.com', 'Web Develop
('Lina Chen', 'lina.chen@example.com', 'UI/UX Design');

-- Students
INSERT OR IGNORE INTO Students (name, email) VALUES
('Noah Kim', 'noah.kim@example.com'),
('Priya Rao', 'priya.rao@example.com'),
('Marcus Taylor', 'marcus.taylor@example.com');

-- Courses
INSERT OR IGNORE INTO Courses (title, description, instructor
('Python Basics', 'Intro to Python', 1),
('Machine Learning', 'ML concepts', 1),
('React Development', 'Frontend with React', 2),
('UI/UX Design', 'Design principles', 3);
```

OUTLINE
TIMELINE
SQLITE EXPLORER

Ln 16, Col 32 Spaces: 4 UTF-8 CRLF {} SQLite Chat quota reached SQLite: database.db Go Live





Explanation:

- Created sample records using SQL INSERT statements for Instructors, Students, and Courses.
- Used INSERT OR IGNORE to avoid duplicate entry errors due to UNIQUE constraints.
- Verified data insertion using SELECT * queries on each table.

- Checked relationships between tables using JOIN queries to ensure correctness.

Task 3: Implement Core Database Operations (CRUD)

Scenario

The platform administrator needs to manage course and student information.

Task Description

Using AI-generated SQL, perform the following operations:

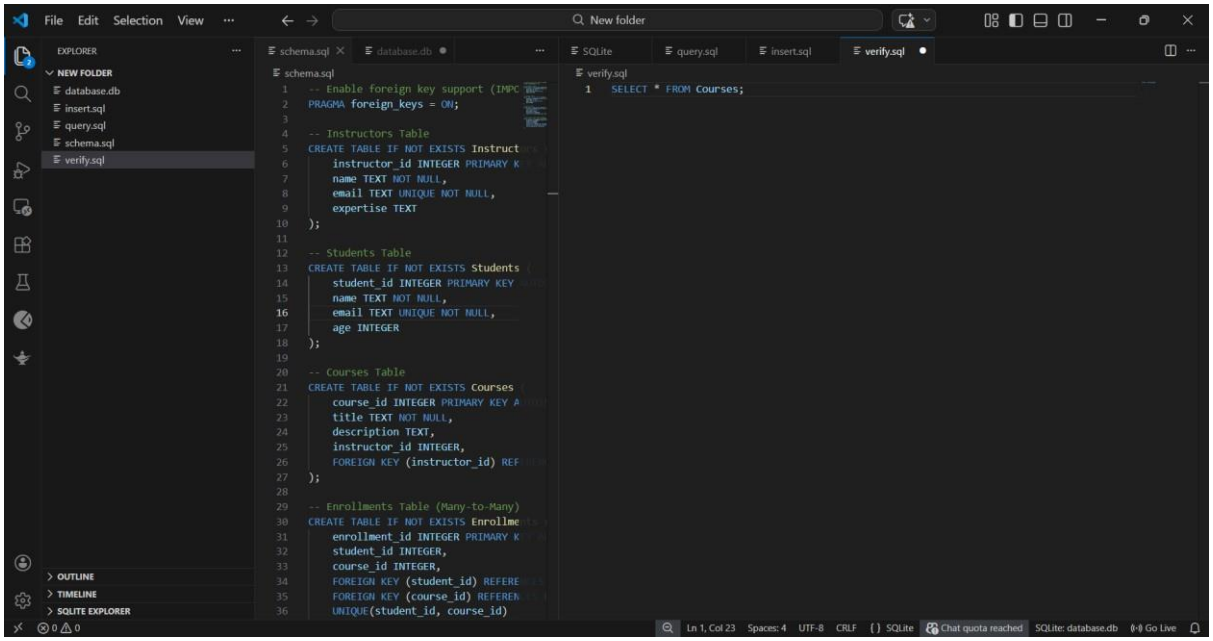
- Retrieve all courses offered
- Update a student's email address
- Remove a course from the database
- Verify each operation using appropriate SELECT statements

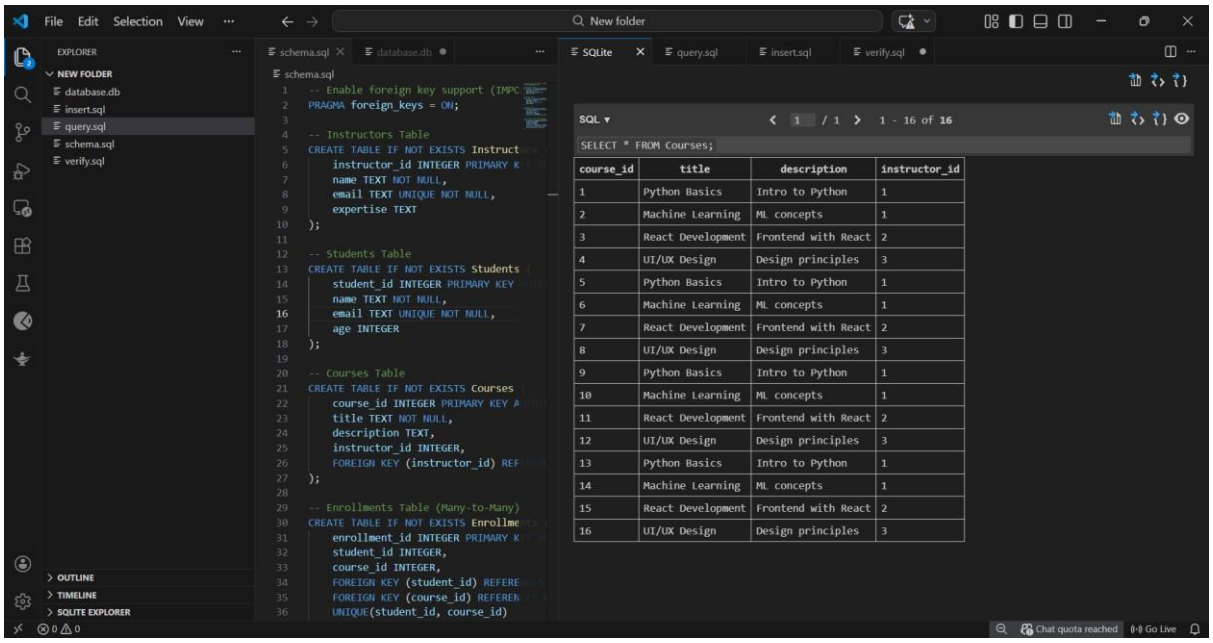
Prompt:

```
Generate SQL queries for:
- Retrieve all courses
- Update student email
- Delete a course
```

Code:

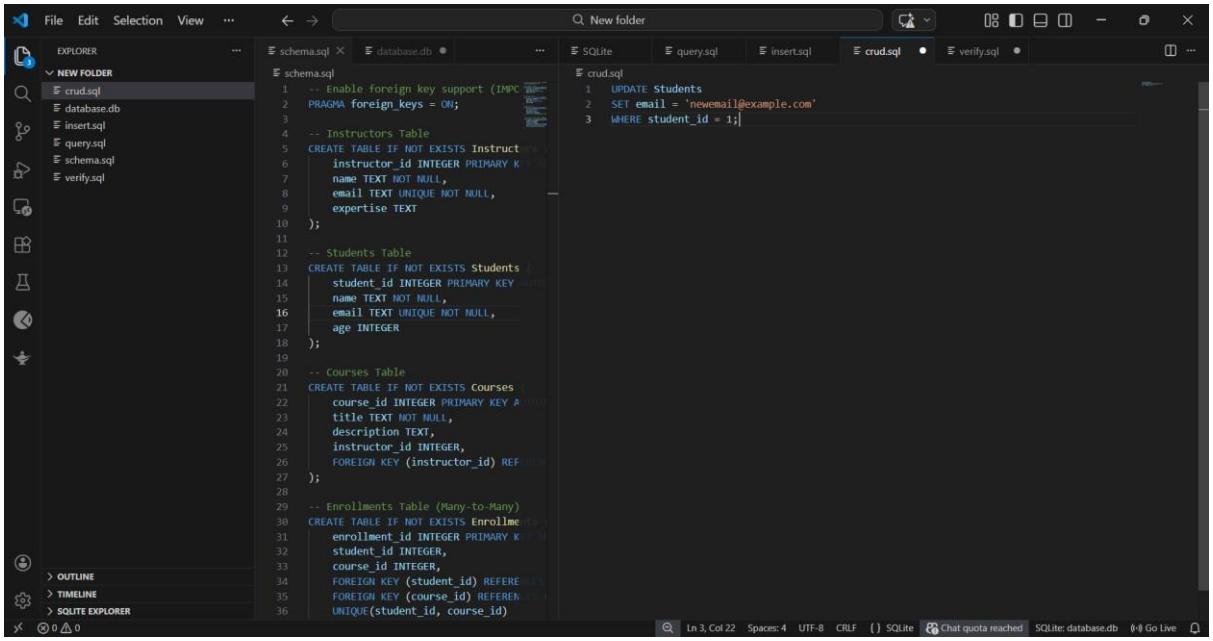
- Retrieve all courses offered

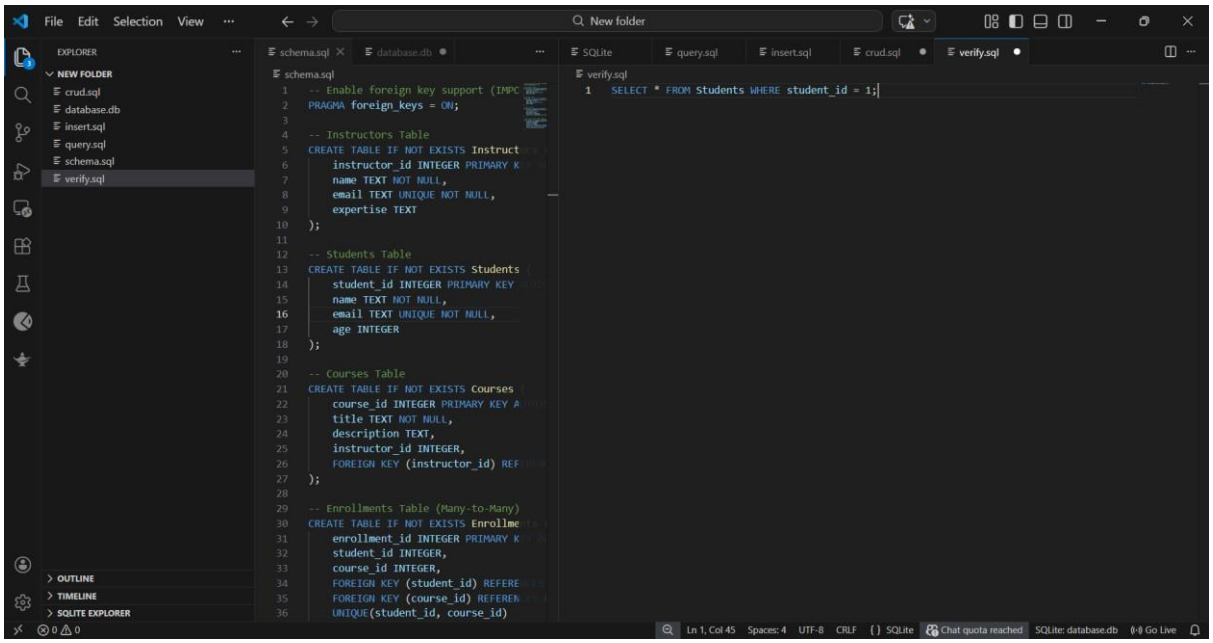


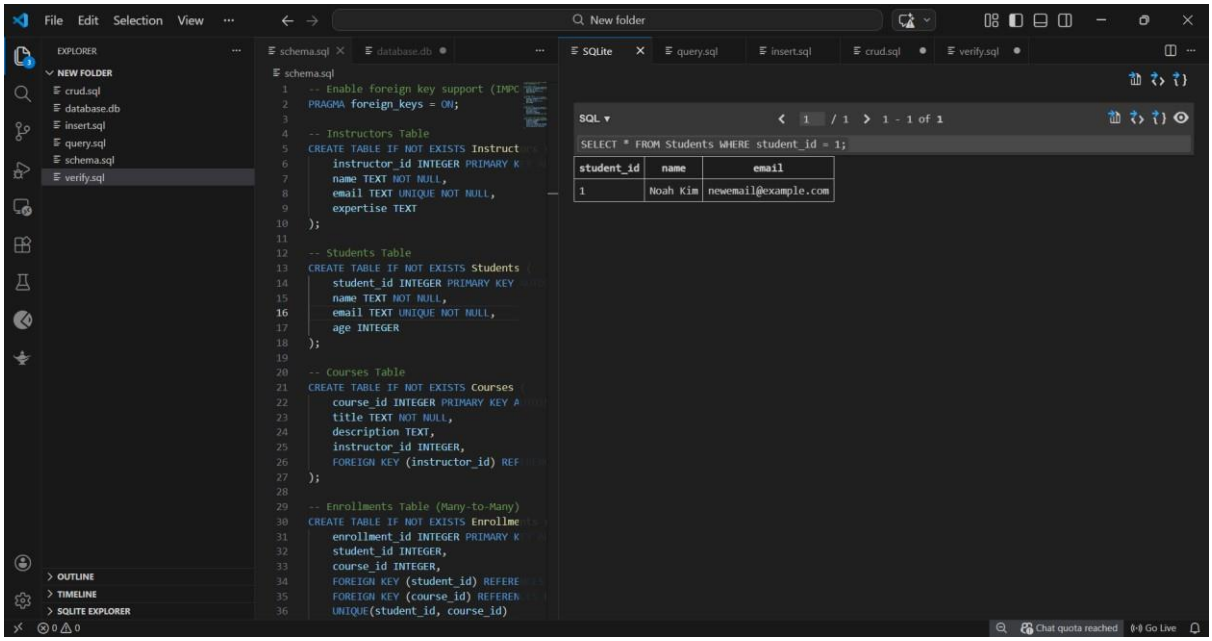


course_id	title	description	instructor_id
1	Python Basics	Intro to Python	1
2	Machine Learning	ML concepts	1
3	React Development	Frontend with React	2
4	UI/UX Design	Design principles	3
5	Python Basics	Intro to Python	1
6	Machine Learning	ML concepts	1
7	React Development	Frontend with React	2
8	UI/UX Design	Design principles	3
9	Python Basics	Intro to Python	1
10	Machine Learning	ML concepts	1
11	React Development	Frontend with React	2
12	UI/UX Design	Design principles	3
13	Python Basics	Intro to Python	1
14	Machine Learning	ML concepts	1
15	React Development	Frontend with React	2
16	UI/UX Design	Design principles	3

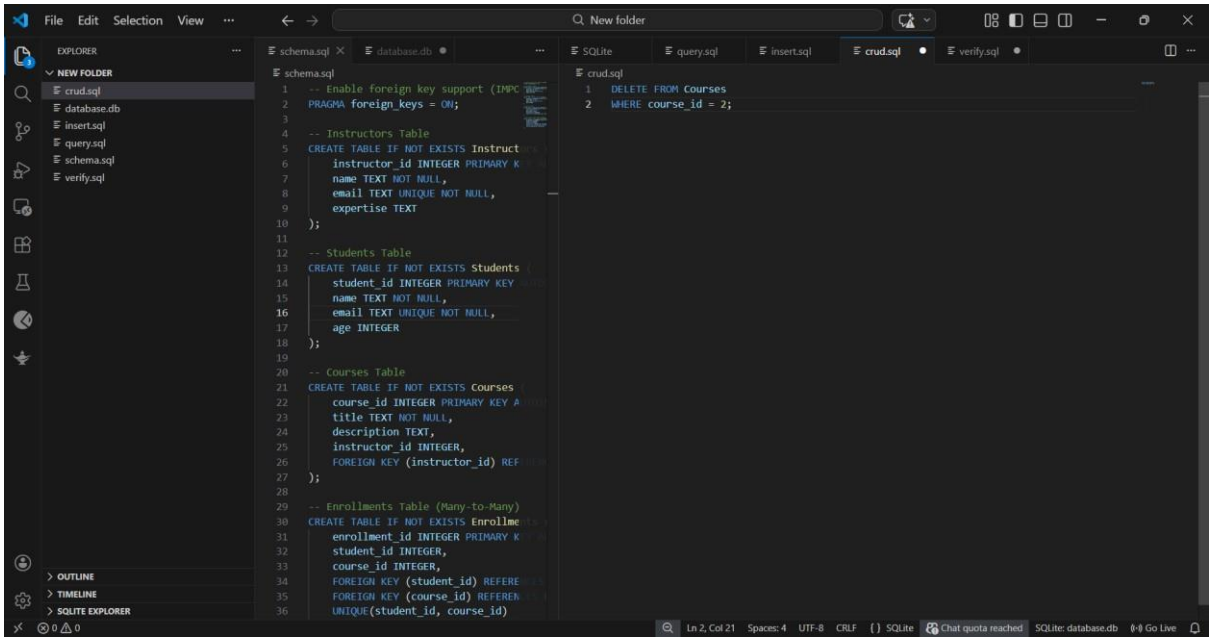
- Update a student’s email address

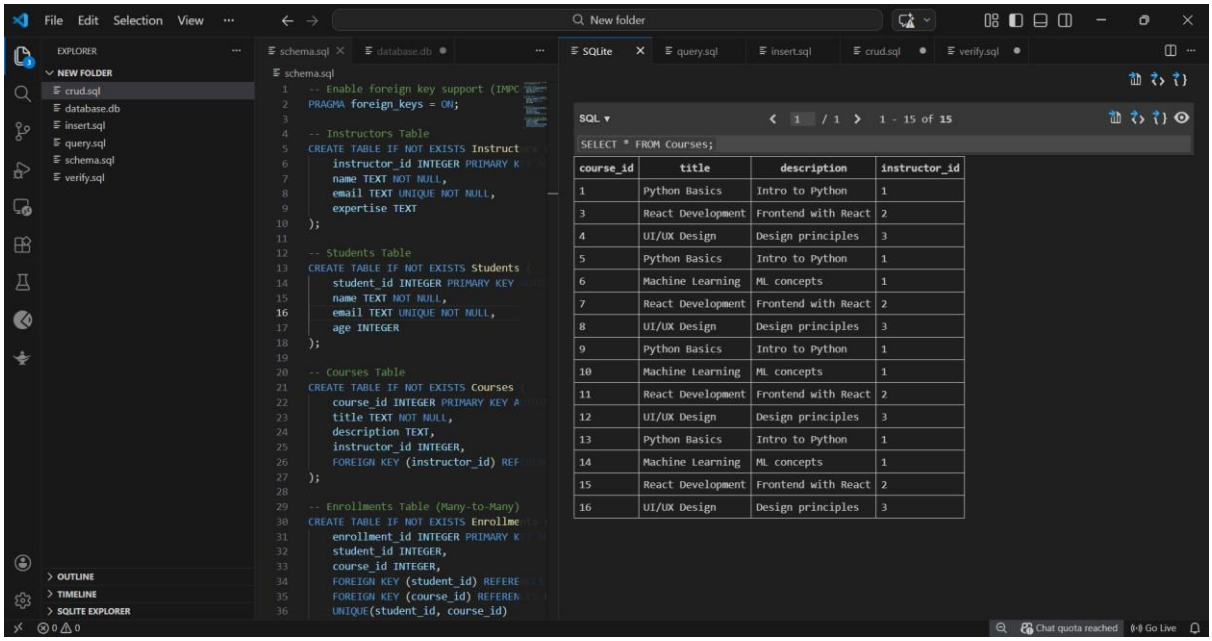

File Edit Selection View
New folder
EXPLORER
NEW FOLDER
crud.sql
database.db
insert.sql
query.sql
schema.sql
verify.sql
OUTLINE
TIMELINE
SQLITE EXPLORER
schema.sql
database.db
SQLite
query.sql
insert.sql
crud.sql
verify.sql
-- Enable foreign key support (IMPC
PRAGMA foreign_keys = ON;
-- Instructors Table
CREATE TABLE IF NOT EXISTS Instruct
instructor_id INTEGER PRIMARY K
name TEXT NOT NULL,
email TEXT UNIQUE NOT NULL,
expertise TEXT
);
-- Students Table
CREATE TABLE IF NOT EXISTS Students
student_id INTEGER PRIMARY KEY
name TEXT NOT NULL,
email TEXT UNIQUE NOT NULL,
age INTEGER
);
-- Courses Table
CREATE TABLE IF NOT EXISTS Courses
course_id INTEGER PRIMARY KEY A
title TEXT NOT NULL,
description TEXT,
instructor_id INTEGER,
FOREIGN KEY (instructor_id) REF
);
-- Enrollments Table (Many-to-Many)
CREATE TABLE IF NOT EXISTS Enrollme
enrollment_id INTEGER PRIMARY K
student_id INTEGER,
course_id INTEGER,
FOREIGN KEY (student_id) REFERE
FOREIGN KEY (course_id) REFEREN
UNIQUE(student_id, course_id)
UPDATE Students
SET email = 'newemail@example.com'
WHERE student_id = 1;
Ln 3, Col 22
Spaces: 4
UTF-8
CRLF
{} SQLite
Chat quota reached
SQLite: database.db
Go Live


File Edit Selection View
New folder
EXPLORER
NEW FOLDER
crud.sql
database.db
insert.sql
query.sql
schema.sql
verify.sql
OUTLINE
TIMELINE
SQLITE EXPLORER
schema.sql
database.db
SQLite
query.sql
insert.sql
crud.sql
verify.sql
-- Enable foreign key support (IMPC
PRAGMA foreign_keys = ON;
-- Instructors Table
CREATE TABLE IF NOT EXISTS Instruct
instructor_id INTEGER PRIMARY K
name TEXT NOT NULL,
email TEXT UNIQUE NOT NULL,
expertise TEXT
);
-- Students Table
CREATE TABLE IF NOT EXISTS Students
student_id INTEGER PRIMARY KEY
name TEXT NOT NULL,
email TEXT UNIQUE NOT NULL,
age INTEGER
);
-- Courses Table
CREATE TABLE IF NOT EXISTS Courses
course_id INTEGER PRIMARY KEY A
title TEXT NOT NULL,
description TEXT,
instructor_id INTEGER,
FOREIGN KEY (instructor_id) REF
);
-- Enrollments Table (Many-to-Many)
CREATE TABLE IF NOT EXISTS Enrollme
enrollment_id INTEGER PRIMARY K
student_id INTEGER,
course_id INTEGER,
FOREIGN KEY (student_id) REFERE
FOREIGN KEY (course_id) REFEREN
UNIQUE(student_id, course_id)
SELECT * FROM Students WHERE student_id = 1;
Ln 1, Col 45
Spaces: 4
UTF-8
CRLF
{} SQLite
Chat quota reached
SQLite: database.db
Go Live



- Remove a course from the database





Explanation:

- Used SQL **SELECT** query to retrieve all courses from the Courses table.
- Performed an **UPDATE** operation to modify a student's email using their student ID.
- Used a **DELETE** query to remove a specific course from the database.
- Verified each operation by running **SELECT queries** to ensure the changes were applied correctly.

Task 4: Reporting Using Aggregate Queries

Scenario

The platform management wants summary reports for decisionmaking.

Task Description

- Use AI tools to generate SQL queries that:

o Count the number of students enrolled in each course o

List courses with more than a specified number of

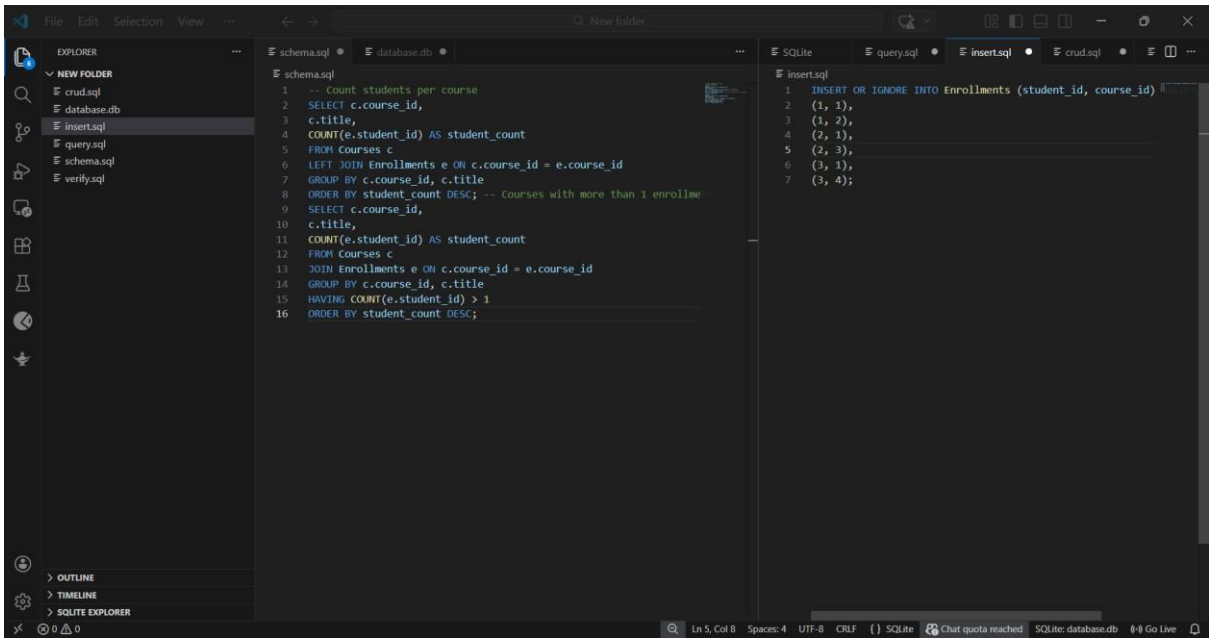
enrollments

- Analyze the query results for correctness.

Prompt:

```
Generate SQL queries to:
- Count students per course
- List courses with more than 1 enrollment
```

Code:



```sql
-- Count students per course
SELECT c.course_id,
c.title,
COUNT(e.student_id) AS student_count
FROM Courses c
LEFT JOIN Enrollments e ON c.course_id = e.course_id
GROUP BY c.course_id, c.title
ORDER BY student_count DESC; -- Courses with more than 1 enrollme
SELECT c.course_id,
c.title,
COUNT(e.student_id) AS student_count
FROM Courses c
JOIN Enrollments e ON c.course_id = e.course_id
GROUP BY c.course_id, c.title
HAVING COUNT(e.student_id) > 1
ORDER BY student_count DESC;
```

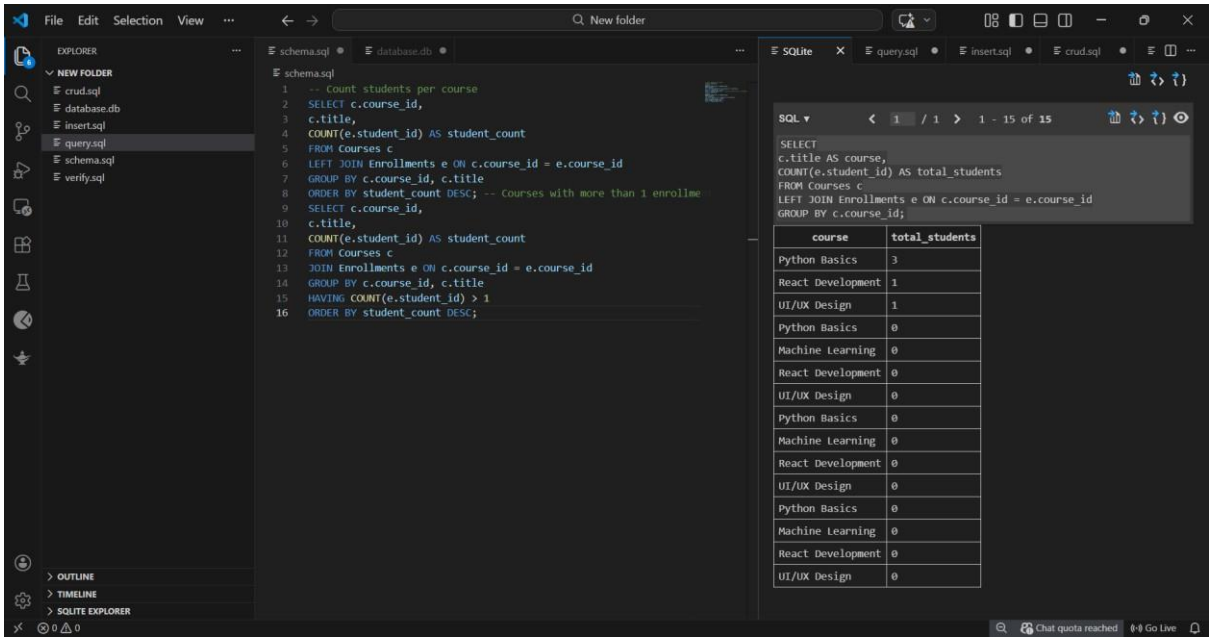
SQL ▾ 1 / 1 1 - 15 of 15

```sql
SELECT c.course_id,
c.title,
COUNT(e.student_id) AS student_count
FROM Courses c
LEFT JOIN Enrollments e ON c.course_id = e.course_id
GROUP BY c.course_id, c.title
ORDER BY student_count DESC;
```

course_id	title	student_count
1	Python Basics	0
3	React Development	0
4	UI/UX Design	0
5	Python Basics	0
6	Machine Learning	0
7	React Development	0
8	UI/UX Design	0
9	Python Basics	0
10	Machine Learning	0
11	React Development	0
12	UI/UX Design	0
13	Python Basics	0
14	Machine Learning	0
15	React Development	0
16	UI/UX Design	0

SQL ▾ 1 / 1 1 - 0 of 0

```sql
SELECT
    c.title AS course,
    COUNT(e.student_id) AS total_students
FROM Courses c
LEFT JOIN Enrollments e ON c.course_id = e.course_id
GROUP BY c.course_id;
```



Explanation:

- Used COUNT() function to calculate number of students per course.
- Applied GROUP BY to group results by course.
- Used HAVING clause to filter courses based on enrollment count.
- Verified results to ensure accurate aggregation and reporting.

Task 5: Query Optimization with AI Suggestions

Scenario

As the platform grows, query performance becomes critical.

Task Description

- Use AI to analyze an existing query that retrieves courses taught by instructors from a specific country.
- Refactor the query using:

o Efficient joins

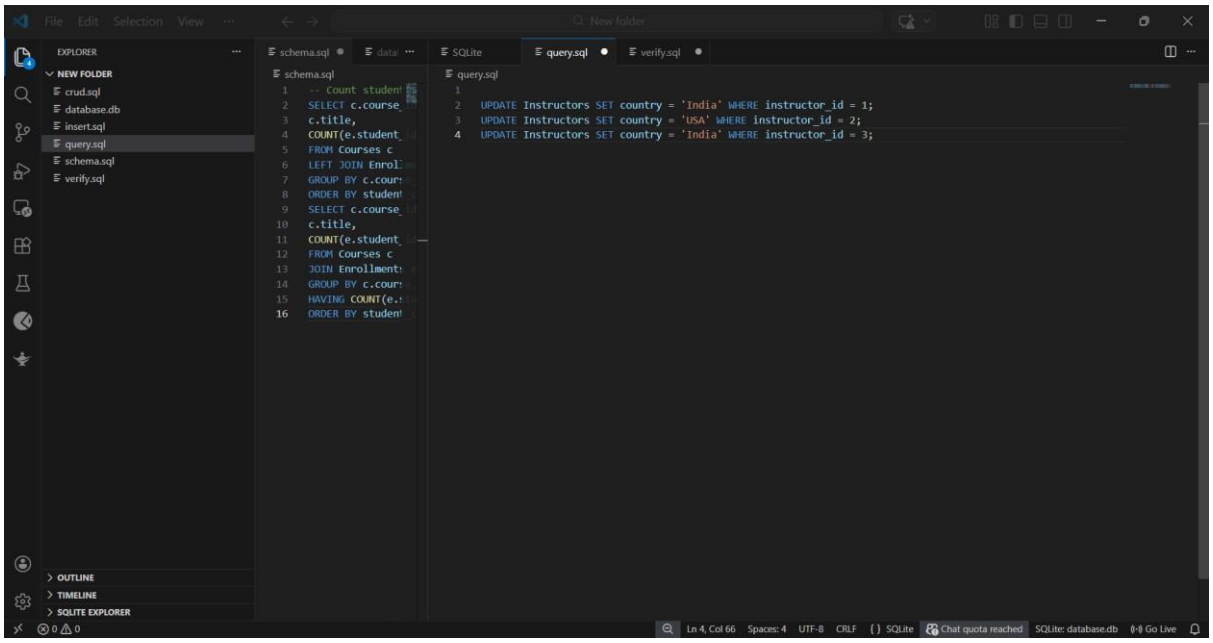
o Optional indexing suggestions

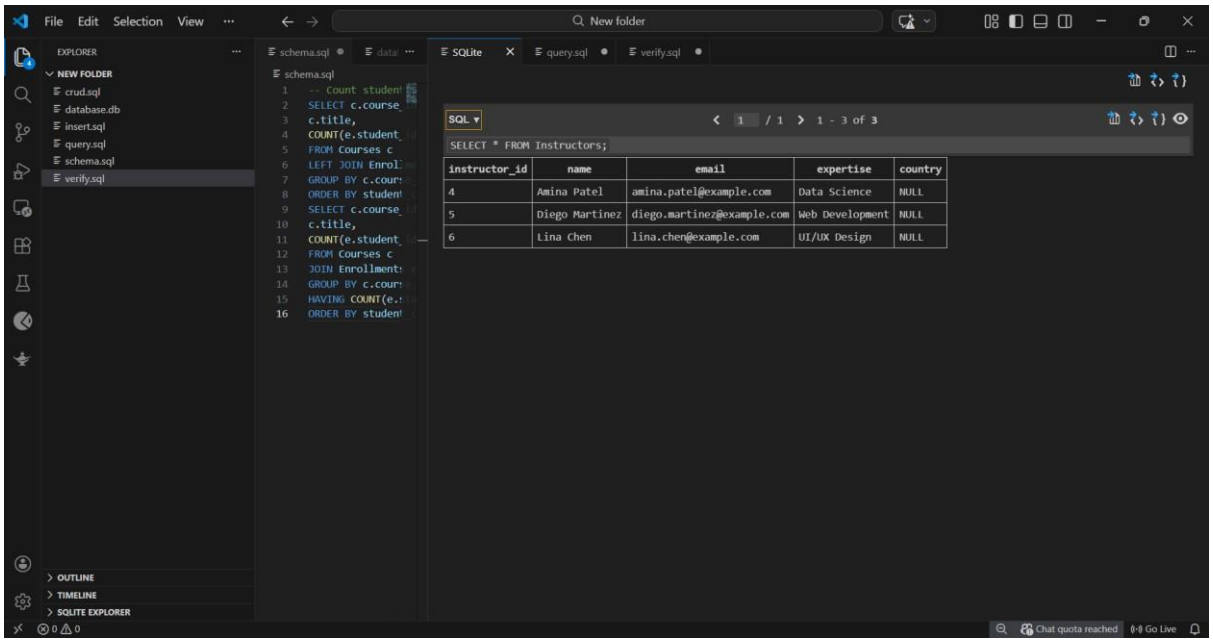
- Compare results before and after optimization.

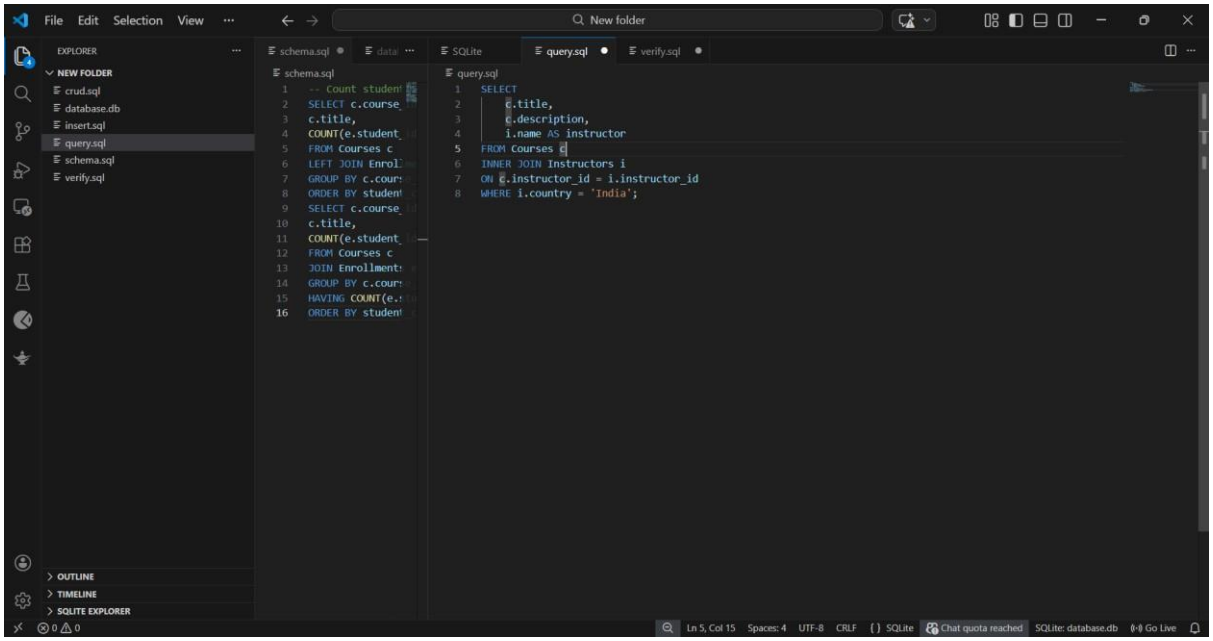
Prompt:

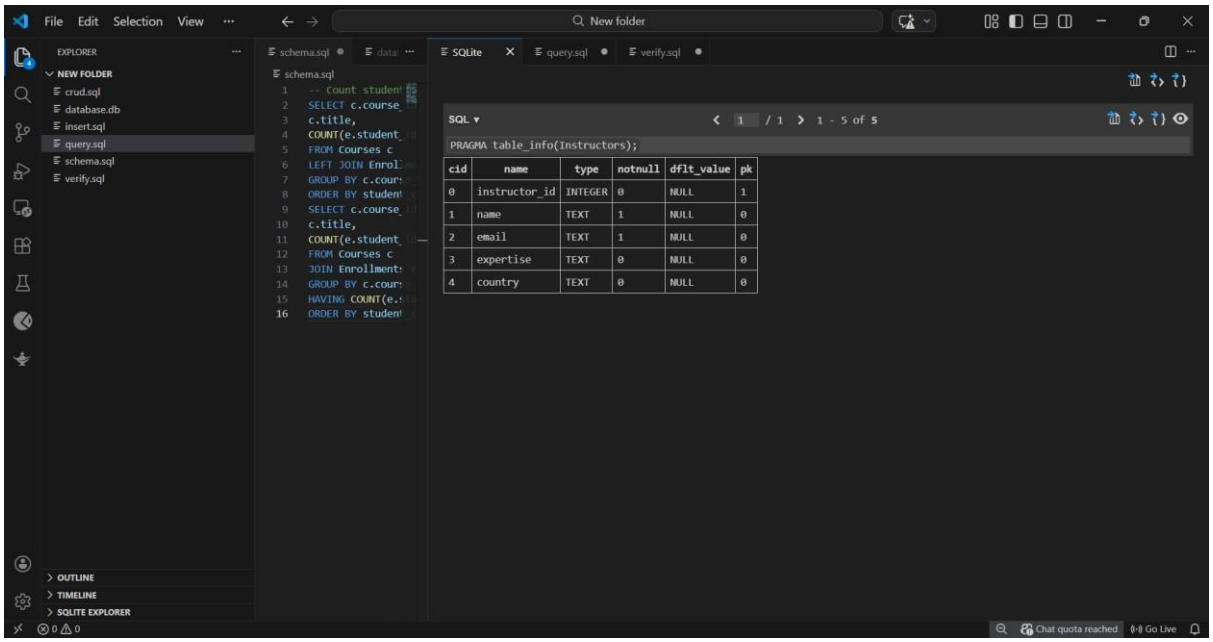
Optimize SQL query to fetch courses taught by instructors from a specific country. Use joins and suggest indexing.

Code:









Explanation:

- Identified error due to missing country column in the Instructors table.
- Added the country column using ALTER TABLE and updated values using UPDATE statements.
- Re-ran the query after fixing the schema to retrieve correct results.